Exam in Concurrent Algorithms and Data Structures

Department of Information Technology
Uppsala University
2024-06-15
Lecturer: Parosh Aziz Abdulla
Location: Bergsbrunnagatan 15, sal 1
Time: 8:00 - 11:00

No books or lecture notes allowed
The use of mobile phones, calculators, and computers is not allowed
Directions:

1. If you are asked to start your answer with one of the words, *yes* or *no*, please do so. Otherwise, you will not get any credit for the answer.

2. All explanations should be concise and to the point. Lengthy explanations might yield a lower grade (and will not give extra points).

Good Luck!

```
add(k):
Node p, c
1  lock(H)
2  p = H
3  c = p.next
4  lock(c)
5  while (c.key < k)
6    unlock(p)
7    p = c
8    c = c.next
9    lock(c)
10 if (c.key = k)
11   return false
12 else
13   n = new Node(k,-)
14   n.next = c
15   p.next = n
16   return true
17 unlock(c)
18 unlock(p)
```

```
rmv(k):
Node p, c
1  lock(H)
2  p = H
3  c = p.next
4  lock(c)
5  while (c.key < k)
6    unlock(p)
7    p = c
8    c = c.next
9    lock(c)
10 if (c.key = k)
11   p.next=c.next
12   return true
13 else
14   return false
15 unlock(c)
16 unlock(p)
```

Figure 1: The fine-grained list algorithm.

Problem 1 Consider the fine-grained algorithm for implementing concurrent sets. Assume that we are only interested in the `add` and the `rmv` operations (i.e., no `ctn` operations). We recall the code in Fig. 1.

- Assume we define Line 13 as the linearization point for a successful `add` operation, and Line 10 as the linearization point for an unsuccessful `add` operation. Is it possible to define lines in `rmv` to act as linearization points for successful resp. unsccessful `rmv` operations? If *yes*, which line and why? If *no*, give the reason.

- Assume we define the linearization points for the `add` as Line 13 (successful) and Line 11 (unsuccessful). Answer the same questions as above.

```
Scan()
1 result:= true
2 for j = n downto 1 do
3     a:= R_j
4     b:=R_{j-1}
5     if not (a = b) then
6         success:= false
7 return(result)
8 exit
```

```
Update(a)
9  n++
10 R_n:= a
11 exit
```

Figure 2: The snapshot library

Problem 2 Recall the `snapshot` abstract data type discussed in the lectures. We consider a different variant here. A state of `snapshot` is an array of length n for some natural number n. The array cells carry natural numbers with an initial value of 0. `snapshot` uses two methods, namely `update`, which increments the size of the array by one and updates the new last cell of the array, and `scan`, which returns the value `true` if and only if all the array elements are identical. The methods are defined through the following two rules:

$$A \xrightarrow{\texttt{update}(a)} A[n+1 := a]$$
$$A \xrightarrow{\texttt{scan}(A)} \texttt{true} \quad \text{if all elements in } A \text{ are identical}$$
$$A \xrightarrow{\texttt{scan}(A)} \texttt{false} \quad \text{there are two distinct elements in } A$$

In Fig. 2, we consider a concurrent library that is supposed to implement `snapshot`. The array is represented by a sequence of registers, where the value of the ith entry is stored in a register $\texttt{R}_i$. All the threads can update and read all the registers.
Answer the following questions:

- Is the library linearizable wrt. `snapshot`, or not? A correct answer to this item only gives credit if one of the following two items is answered correctly.
- If *yes*, give a precise linearization policy. Vague answers will not give any credit.

```
enq(k):
Node t,x
1  n = new Node(k,NULL)
2  while (true)
3    t = Tail
4    x = t.next
5    if (t == Tail)
6      if (x == NULL)
7        if (CAS (t.next,NULL,n))
8          CAS (Tail,t,n)
9          exit
10     else
11       CAS (Tail,t,x)
```

original version

```
deq(k):
Node h,t,x
Integer v
1  while (true)
2    h = Head
3    t = Tail
4    x = h.next
5    if (h == Head)
6      if (h == t)
7        if (x == NULL)
8          return *
9          exit
10       CAS (Tail,t,x)
11     else
12       v = x.key
13       if (CAS (Head,h,x))
14         return v
15         exit
```

```
enq(k):
Node t,x
1  n = new Node(k,NULL)
2  while (true)
3    t = Tail
4    x = t.next
5    if (t == Tail)
6      if (x == NULL)
7        if (CAS (t.next,NULL,n))
8          CAS (Tail,t,n)
9          exit
10
11
```

compact version

Figure 3: The new version of the Michel-Scott algorithm.

- If *no*, give a concurrent history demonstrating this. To get credits, you need to describe (i) how the library generated the history and (ii) why the history is not linearizable wrt. `snapshot`.

Problem 3 Conisder the new version of the Michael-Scott algorithm depicted in Fig. 3. In the new algorithm we keep the dequeue part, but replace the enqueue part by a more compact code where we have removed the lines 10 and 11.
Is the resulting algorithm linearizable? First, answer *yes* or *no*. If *yes*, give the linearization policy, and motivate your answer in no more than five lines. If *no* give a concurrent history that shows non-linearizability.

Problem 4 Consider the following algorithm that is supposed to implement a set. The algorithm modifies the Lazy List algorithm we studied in the

```
locate(e)
   local p, c, n, m, s;
1  while (true) {
2      p := Head;
3      c := p.next;
4      while (true) {
5          (n, m) := c.(next, mark);
6          while (m) {
7              s := cas(&(p.(next, mark)), c, n, false, false);
8              if (!s) goto line 1;
9              c:=n;
10             (n, m) := c.(next, mark);
11         }
12         if (c.val >= e)
13             return (p, c);
14         p:=c;
15         c:=n;
16     }
17 }
```

```
add(e)
   local p, c, n;
18 while (true) {
19     (p, c) := locate(e);
20     if (c.val != e) {
21     n := new Node (e, c, false);
22     if (cas(&(p.(next, mark)), c, n, false, false))
23         return true;
24     } else {
25         return false;
26     }
27 }
```

Figure 4: Part 1 of the Algorithm of Problem 4.

lectures. The main difference is using `cas` instructions instead of locks. Recall the classical definition of `cas` (a, b, c) where we *atomically* (i) compare a and b, (ii) if $a = b$ then assign c to a, and return `true`, and (iii) if $a \neq b$ then do not make any assignments, and return `false`. The algorithm uses an extended `cas` instruction with five parameters, where we use `cas` $(a_1, a_2, b_1, c_1, b_2, c_2)$ to *atomically* (i) compare a_1 and b_1, (ii) compare a_2 and b_2 , (ii) if both $a_1 = b_1$ and $a_2 = b_2$ then assign c_1 to a_1, assign c_2 to a_2, and return `true`, and (iii) if $a_1 \neq b_1$ or $a_2 \neq b_2$ then do not make any assignments, and return `false`. In particular

$$\texttt{cas}(\&(\texttt{p.}(\texttt{next}, \texttt{mark})), \texttt{o}, \texttt{n}, \texttt{b}, \texttt{b}')$$

updates `p.next` and `p.mark` atomically to `n` and `b'`, if they were originally `o` and `b` respectively.
The following command allows reading p.next and p.mark atomically:

$$(\texttt{n}, \texttt{b}) := \texttt{p.}(\texttt{next}, \texttt{mark});$$

In other words, we treat the node's `next` and `mark` fields as a single atomic unit. The algorithm only updates the next field when the mark field is true. Is the set implementation shown in Fig. 4 and Fig. 5 linearizable? Your answer needs to be motivated as explained in Problem 2.

```
rmv(e)
    local p, c, n, s;
28  while (true) {
29      (p, c) := locate(e);
30      if(c.val = e){
31          n := c.next;
32          s := cas(&(c.(next, mark)), n, n, false, true);
33          if (!s)
34              goto line 28;
35          cas(&(p.(next, mark)), c, n, false, false);
36          return true;
37      } else {
38          return false;
39      }
40  }
```

```
ctn(e)
    local c
41  c := Head;
42  while (c.val < e) {
43      c := c.next;
44  }
45  b := c.mark;
46  if (!b && c.val=e )
47      return true;
48  else
49      return false;
```

Figure 5: Part 2 of the Algorithm of Problem 4.